NAME :- Lokare Mahir Abdul Majeed

ROLL NO :- 23AI34

BATCH :- 02

EXPERIMENT NO :- 04

AIM :-

To design and implement a fully connected Deep Neural Network for a classification problem.

OBJECTIVE :-

1. To understand the concept of fully connected Deep Neural Networks.
2. To design a DNN using fully connected Dense layers.
3. To understand the use of ReLU activation in hidden layers.
4. To use Softmax activation for multi-class classification.
5. To train the DNN using the MNIST dataset.
6. To evaluate the performance of the trained model using accuracy and loss.

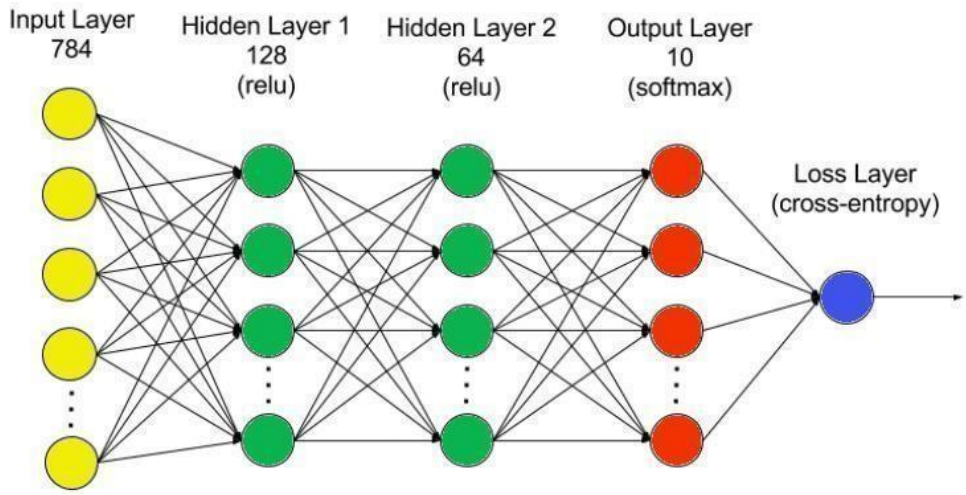
THEORY :-

Fully Connected Deep Neural Network :-

A Fully Connected Neural Network, also called a Dense Neural Network, is a neural network in which every neuron of one layer is connected to every neuron of the next layer. Fully connected layers use weights and biases to transform the input data and learn patterns during training.

A DNN contains multiple layers between the input and output layers. For a classification problem, the hidden layers learn useful representations of the input data, while the final output layer produces the class probabilities.

Architecture used in this experiment:



Fully Connected Layer :-

A fully connected layer is a layer in which every neuron is connected to every neuron in the previous layer. These connections contain weights that are learned during training. Bias values are also added to the weighted sum.

The operation of a fully connected layer can be represented as: $Z = XW + b$

Where: X is the input, W represents the weights, b represents the bias, Z is the weighted output.

ReLU Activation Function :-

ReLU stands for Rectified Linear Unit. It introduces non-linearity into the neural network and is commonly used in hidden layers.

ReLU(x) = max(0, x)

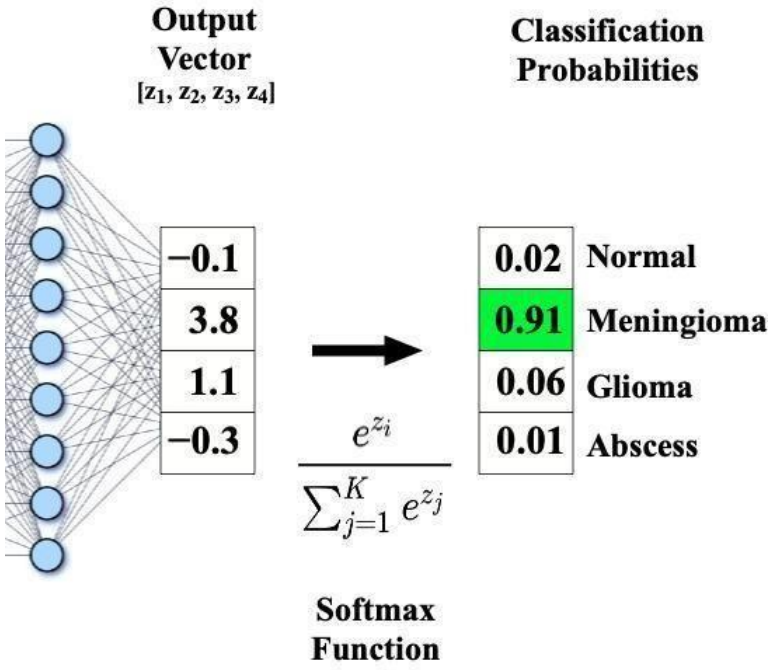
For positive values, ReLU returns the input value, while for negative values it returns zero. ReLU helps the network learn complex patterns and provides effective gradient flow during training.

Softmax Activation Function :-

The Softmax function is used in the output layer for multi-class classification. It converts the output values of the network into probabilities for each class. The probabilities lie between 0 and 1 and their sum is equal to 1.

The Softmax function is given by: Softmax(zi) = e^zi / Σe^zj

Where: zi is the output value for class i, e is the exponential function, and the denominator represents the sum of exponentials of all class outputs.



Loss Function :-

For multi-class classification, categorical cross-entropy is commonly used with Softmax output. It measures the difference between the actual class and the predicted probability distribution.

The categorical cross-entropy loss is: L = −Σ yi log(ŷi)

Where: yi is the actual class value, ŷi is the predicted probability for class i. A lower loss indicates that the predicted probabilities are closer to the actual class labels.

Classification Process :-

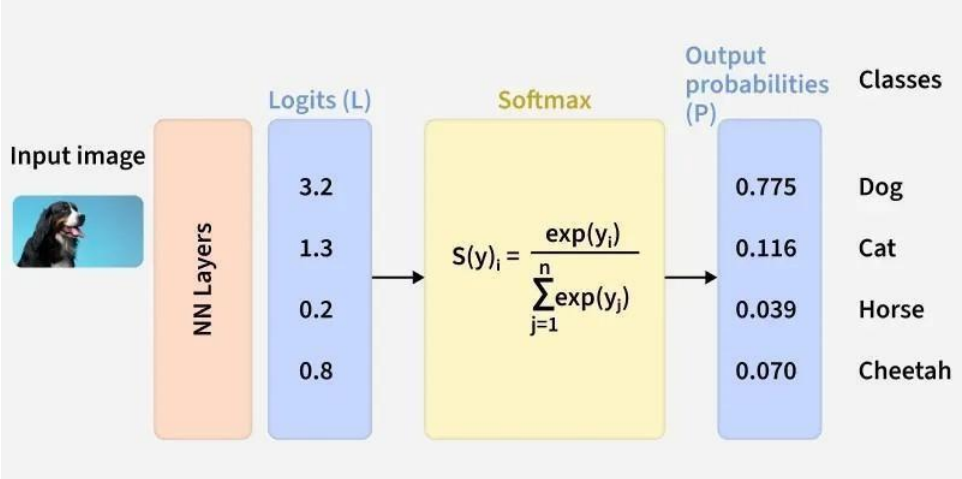
The classification process consists of the following steps:

1. The MNIST image is provided as input.

2. The image is flattened into a one-dimensional vector.

3. The first fully connected layer learns features using ReLU activation.

4. The second fully connected layer learns higher-level patterns.
5. The output layer uses Softmax to calculate probabilities for all 10 classes.
6. The class having the highest probability is selected as the predicted digit.



PROCEDURE :-

1. Import TensorFlow and Matplotlib.
2. Load the MNIST dataset.
3. Normalize the pixel values between 0 and 1.
4. Create a fully connected DNN using Dense layers.
5. Flatten the 28 × 28 input images.
6. Add the first Dense layer with 128 neurons and ReLU activation.
7. Add the second Dense layer with 64 neurons and ReLU activation.
8. Add the output layer with 10 neurons and Softmax activation.
9. Compile the model using Adam optimizer and sparse categorical cross-entropy loss.
10. Train the model using the training dataset.
11. Evaluate the model using the test dataset.
12. Display the test accuracy and loss.
13. Plot the training and validation accuracy.
14. Display sample predictions from the test dataset.

CODE :-

```
import tensorflow as tf
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
```

```
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.2
)

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

predictions = model.predict(
    x_test[:10],
    verbose=0
)

predicted_classes = predictions.argmax(axis=1)

print("Predicted Classes:", predicted_classes)
print("Actual Classes:", y_test[:10])

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

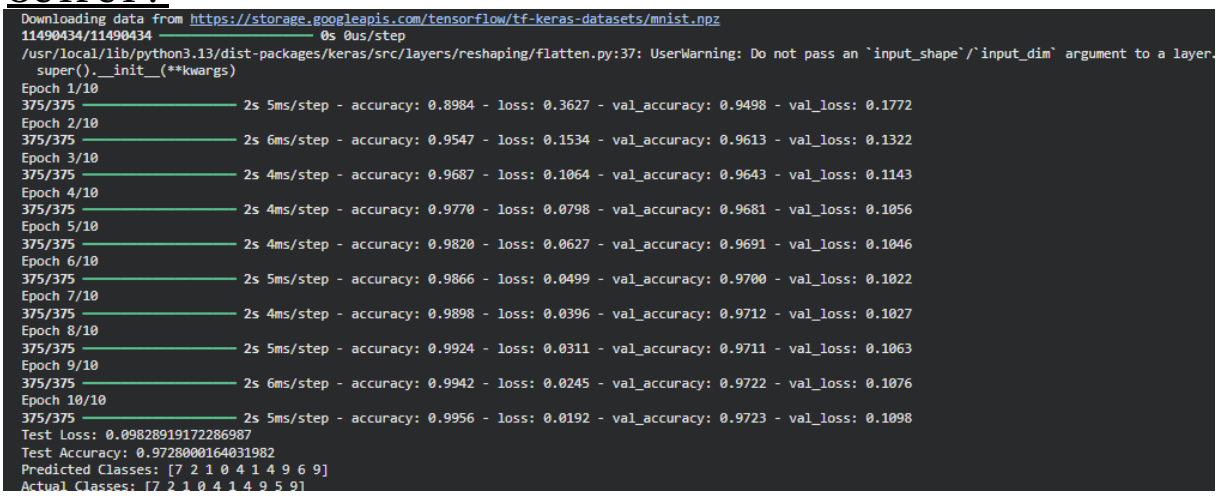
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

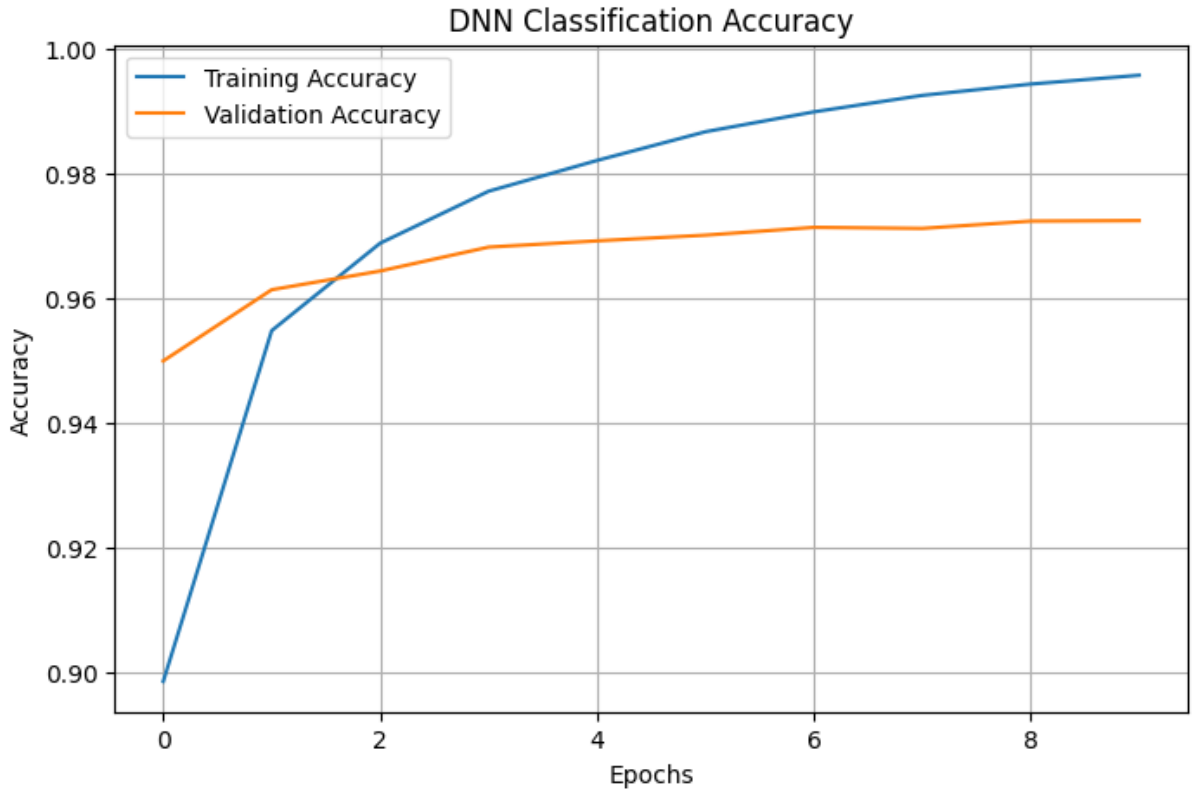
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("DNN Classification Accuracy")
```

```
    plt.legend()
    plt.grid(True)

plt.show()
```

OUTPUT :-





CONCLUSION :-

The experiment successfully implemented a fully connected DNN for MNIST classification using Dense layers with ReLU and Softmax activation. The model was trained using Adam optimizer and sparse categorical cross-entropy loss and successfully classified handwritten digits.